```
"use strict";

// ========================================
// AUDIOBOOK ORB — INTERACTIVE VISUAL PROTOTYPE
// ========================================

const $ = (selector) => document.querySelector(selector);
const $$ = (selector) => document.querySelectorAll(selector);

const body = document.body;

const playButton = $("#playButton");
const playIcon = $("#playIcon");
const rewindButton = $("#rewindButton");
const restartButton = $("#restartButton");
const signalButton = $("#signalButton");

const progressTrack = $("#progressTrack");
const progressFill = $("#progressFill");
const progressKnob = $("#progressKnob");
const elapsedTime = $("#elapsedTime");
const durationTime = $("#durationTime");

const sessionStatus = $("#sessionStatus");
const orbCaption = $("#orbCaption");
const orbHeading = $("#orbHeading");
const modeTag = $("#modeTag");

const behaviourNote = $("#behaviourNote");
const behaviourButtons = $$(".behaviour-option");

const intensityInput = $("#intensity");
const intensityValue = $("#intensityValue");
const breathSpeedInput = $("#breathSpeed");
const breathValue = $("#breathValue");

const settingsButton = $("#settingsButton");
const settingsDialog = $("#settingsDialog");
const closeSettings = $("#closeSettings");
const doneSettings = $("#doneSettings");
const reducedMotionInput = $("#reducedMotion");

const toast = $("#toast");

const SESSION_LENGTH = 25 * 60;

// Current prototype state.
let orbState = "breathing";
let isPlaying = true;
let narrationEnabled = true;
```

```
let reducedMotion = false;

let elapsed = 0;
let intensity = Number(intensityInput.value) / 100;
let breathDuration = Number(breathSpeedInput.value);

let voiceLevel = 0;
let previousOrbTime = null;
let previousSessionTime = null;

let orbFrame = null;
let sessionFrame = null;
let toastTimeout = null;

// The simulation uses a repeating speaking/silence pattern.
const SPEECH_DURATION = 8;
const SILENCE_DURATION = 4;
const SPEECH_CYCLE = SPEECH_DURATION + SILENCE_DURATION;

// ========================================
// HELPERS
// ========================================

function formatTime(seconds) {
  const value = Math.floor(seconds);
  const minutes = Math.floor(value / 60);
  const remaining = value % 60;

  return (
    String(minutes).padStart(2, "0") +
    ":" +
    String(remaining).padStart(2, "0")
  );
}

function smoothToward(current, target, delta, speed) {
  const amount = 1 - Math.exp(-delta * speed);
  return current + (target - current) * amount;
}

function showToast(message) {
  toast.textContent = message;
  toast.classList.add("visible");

  clearTimeout(toastTimeout);

  toastTimeout = setTimeout(() => {
    toast.classList.remove("visible");
  }, 2300);
}
```

```
// ========================================
// THREE ORB BEHAVIOURS
// ========================================

const behaviourInfo = {
 source: {
  title: "A precise source of light",
  tag: "LIGHT SOURCE",
  caption: "LIGHT, EMITTED FROM THE CENTRE",
  note: "A tiny central point emits light through the glass, creating a delicate horizontal flare."
 },

 breathing: {
  title: "A breathing presence",
  tag: "BREATHING",
  caption: "LIGHT, HELD IN DARKNESS",
  note: "The orb breathes independently. The simulated light fades during pauses and returns
with the narration."
 },

 silence: {
  title: "Resting in silence",
  tag: "SILENCE",
  caption: "RESTING IN BLACK",
  note: "The stage becomes completely black. The orb's breathing continues invisibly
underneath."
 }
};

function setOrbState(nextState) {
 if (!behaviourInfo[nextState]) return;

 orbState = nextState;

 body.dataset.orbState = nextState;

 behaviourButtons.forEach((button) => {
  const active = button.dataset.orbState === nextState;

  button.classList.toggle("active", active);
  button.setAttribute("aria-pressed", String(active));
 });

 const info = behaviourInfo[nextState];

 orbHeading.textContent = info.title;
 modeTag.textContent = info.tag;
 behaviourNote.textContent = info.note;

 if (!isPlaying) {
  orbCaption.textContent = "PREVIEW PAUSED";
```

```
  } else {
    orbCaption.textContent = info.caption;
  }

  showToast(`${info.tag.charAt(0) + info.tag.slice(1).toLowerCase()} preview`);
}

behaviourButtons.forEach((button) => {
  button.addEventListener("click", () => {
    setOrbState(button.dataset.orbState);
  });
});

// ========================================
// SIMULATED NARRATOR
// ========================================

// A gentle test signal, not real audio analysis.
// It has no abrupt pulses and makes no claim to detect emotion.

function getNarratorSignal(time) {
  if (!narrationEnabled) return 0;

  const position = time % SPEECH_CYCLE;

  if (position >= SPEECH_DURATION) {
    return 0;
  }

  const fadeIn = Math.min(position / 1.5, 1);

  const fadeOut = Math.min(
    (SPEECH_DURATION - position) / 1.7,
    1
  );

  const variation =
    0.78 +
    0.10 * Math.sin(time * 1.05) +
    0.06 * Math.sin(time * 0.48 + 1.7);

  return Math.max(
    0,
    Math.min(1, fadeIn * fadeOut * variation)
  );
}

// Soft colour transitions between blue, muted green and restrained red.
function getLightColour(time) {
  const colours = [
    [76, 151, 255],
```

```
  [86, 159, 132],
  [171, 91, 103]
 ];

 const cycle = (time / 28) % colours.length;
 const index = Math.floor(cycle);
 const fraction = cycle - index;

 const eased = fraction * fraction * (3 - 2 * fraction);

 const start = colours[index];
 const end = colours[(index + 1) % colours.length];

 return start.map((value, i) =>
  Math.round(value + (end[i] - value) * eased)
 ).join(", ");
}

// ========================================
// ORB ANIMATION
// ========================================

function animateOrb(timestamp) {
 if (previousOrbTime === null) {
  previousOrbTime = timestamp;
 }

 // Limit large time steps after a background-tab pause.
 const delta = Math.min(
  (timestamp - previousOrbTime) / 1000,
  0.1
 );

 previousOrbTime = timestamp;

 const time = timestamp / 1000;

 let targetLevel = 0;

 if (isPlaying) {
  if (orbState === "source") {
   // The source mode holds a stable central light.
   targetLevel = 0.95;
  } else if (orbState === "breathing") {
   // The light follows our simulated narration.
   targetLevel = getNarratorSignal(time);
  } else {
   // Silence is absolute black.
   targetLevel = 0;
  }
 }
```

```
 // Light appears gently and fades away more slowly.
 voiceLevel = smoothToward(
  voiceLevel,
  targetLevel,
  delta,
  targetLevel > voiceLevel ? 1.9 : 1.05
 );

 // The intensity setting controls the available brightness range.
 const lightLevel = Math.min(1, voiceLevel * intensity);

 body.style.setProperty(
  "--light-level",
  lightLevel.toFixed(4)
 );

 body.style.setProperty(
  "--voice-color",
  getLightColour(time)
 );

 // Explicit silence state, with a small threshold to avoid
 // leaving a faint residual edge on screen.
 body.dataset.silent = String(lightLevel < 0.008);

 // Update the caption without competing with the light.
 if (!isPlaying) {
  orbCaption.textContent = "PREVIEW PAUSED";
 } else if (orbState === "silence") {
  orbCaption.textContent = behaviourInfo.silence.caption;
 } else if (orbState === "source") {
  orbCaption.textContent = behaviourInfo.source.caption;
 } else if (lightLevel < 0.008) {
  orbCaption.textContent = "RESTING IN SILENCE";
 } else {
  orbCaption.textContent = "LIGHT, HELD IN DARKNESS";
 }

 orbFrame = requestAnimationFrame(animateOrb);
}

// ========================================
// SESSION PROGRESS
// ========================================

function updateProgress() {
 const progress = Math.min(
  100,
  (elapsed / SESSION_LENGTH) * 100
 );
```

```
  progressFill.style.width = `${progress}%`;
  progressKnob.style.left = `${progress}%`;

  elapsedTime.textContent = formatTime(elapsed);

  progressTrack.setAttribute(
    "aria-valuenow",
    String(Math.round(progress))
  );

  progressTrack.setAttribute(
    "aria-valuetext",
    `${formatTime(elapsed)} of ${formatTime(SESSION_LENGTH)}`
  );
}

function sessionTick(timestamp) {
  if (!isPlaying) {
    sessionFrame = null;
    return;
  }

  if (previousSessionTime === null) {
    previousSessionTime = timestamp;
  }

  const delta = Math.min(
    (timestamp - previousSessionTime) / 1000,
    0.5
  );

  previousSessionTime = timestamp;

  elapsed = Math.min(
    SESSION_LENGTH,
    elapsed + delta
  );

  updateProgress();

  if (elapsed >= SESSION_LENGTH) {
    elapsed = SESSION_LENGTH;
    isPlaying = false;
    sessionFrame = null;
    previousSessionTime = null;

    updatePlaybackUI();
    showToast("Your demo session is complete.");
    return;
  }
```

```
  sessionFrame = requestAnimationFrame(sessionTick);
}

function startSession() {
  if (!isPlaying || sessionFrame !== null) return;

  previousSessionTime = null;
  sessionFrame = requestAnimationFrame(sessionTick);
}

function stopSession() {
  if (sessionFrame !== null) {
    cancelAnimationFrame(sessionFrame);
    sessionFrame = null;
  }

  previousSessionTime = null;
}

// ========================================
// PLAYBACK CONTROLS
// ========================================

function updatePlaybackUI() {
  playIcon.textContent = isPlaying ? " ⏸ " : "▶";

  playButton.setAttribute(
    "aria-label",
    isPlaying ? "Pause preview" : "Resume preview"
  );

  sessionStatus.textContent = isPlaying
    ? "PREVIEW PLAYING"
    : "PREVIEW PAUSED";

  body.dataset.paused = String(!isPlaying);

  if (isPlaying) {
    startSession();
  } else {
    stopSession();
  }
}

playButton.addEventListener("click", () => {
  if (elapsed >= SESSION_LENGTH) {
    elapsed = 0;
    updateProgress();
  }
```

```
 isPlaying = !isPlaying;
 updatePlaybackUI();
});

rewindButton.addEventListener("click", () => {
 elapsed = Math.max(0, elapsed - 15);
 updateProgress();
 showToast("Moved back 15 seconds.");
});

restartButton.addEventListener("click", () => {
 elapsed = 0;
 updateProgress();

 isPlaying = true;
 updatePlaybackUI();

 showToast("Preview restarted.");
});

// The simulated voice can be stopped independently of the session.
signalButton.addEventListener("click", () => {
 narrationEnabled = !narrationEnabled;

 signalButton.textContent = narrationEnabled
  ? "Pause voice"
  : "Resume voice";

 showToast(
  narrationEnabled
   ? "Simulated narration resumed."
   : "Simulated narration paused."
 );
});

// Click or tap the timeline to seek.
function seekToPosition(clientX) {
 const rect = progressTrack.getBoundingClientRect();

 if (!rect.width) return;

 const fraction = Math.max(
  0,
  Math.min(1, (clientX - rect.left) / rect.width)
 );

 elapsed = fraction * SESSION_LENGTH;
 updateProgress();

 if (elapsed >= SESSION_LENGTH) {
  elapsed = SESSION_LENGTH;
```

```
    isPlaying = false;
    updatePlaybackUI();
  }
}

progressTrack.addEventListener("click", (event) => {
  seekToPosition(event.clientX);
});

progressTrack.addEventListener("keydown", (event) => {
  if (event.key === "ArrowRight") {
    event.preventDefault();
    elapsed = Math.min(SESSION_LENGTH, elapsed + 15);
  } else if (event.key === "ArrowLeft") {
    event.preventDefault();
    elapsed = Math.max(0, elapsed - 15);
  } else if (event.key === "Home") {
    elapsed = 0;
  } else if (event.key === "End") {
    elapsed = SESSION_LENGTH;
  } else {
    return;
  }

  updateProgress();

  if (elapsed >= SESSION_LENGTH) {
    isPlaying = false;
    updatePlaybackUI();
  }
});

// ========================================
// VISUAL SETTINGS
// ========================================

intensityInput.addEventListener("input", () => {
  intensity = Number(intensityInput.value) / 100;

  intensityValue.textContent = `${intensityInput.value}%`;
  body.style.setProperty("--light-level", intensity);
});

breathSpeedInput.addEventListener("input", () => {
  breathDuration = Number(breathSpeedInput.value);

  body.style.setProperty(
    "--breath-duration",
    `${breathDuration}s`
  );
```

```
  breathValue.textContent =
    breathDuration >= 15
      ? "Very slow"
      : breathDuration >= 11
        ? "Slow"
        : breathDuration >= 8
          ? "Gentle"
          : "Faster";
});

// ========================================
// SETTINGS DIALOG
// ========================================

settingsButton.addEventListener("click", () => {
  if (typeof settingsDialog.showModal === "function") {
    settingsDialog.showModal();
  }
});

function closeSettingsDialog() {
  if (settingsDialog.open) {
    settingsDialog.close();
  }
}

closeSettings.addEventListener("click", closeSettingsDialog);
doneSettings.addEventListener("click", closeSettingsDialog);

settingsDialog.addEventListener("click", (event) => {
  if (event.target === settingsDialog) {
    closeSettingsDialog();
  }
});

function applyReducedMotion(enabled) {
  reducedMotion = enabled;

  body.classList.toggle("reduced-motion", reducedMotion);
}

reducedMotionInput.addEventListener("change", () => {
  applyReducedMotion(reducedMotionInput.checked);
});

// Respect the operating system's accessibility preference.
const motionPreference = window.matchMedia(
  "(prefers-reduced-motion: reduce)"
);

applyReducedMotion(motionPreference.matches);
```

```
reducedMotionInput.checked = motionPreference.matches;

if (motionPreference.addEventListener) {
  motionPreference.addEventListener("change", (event) => {
    reducedMotionInput.checked = event.matches;
    applyReducedMotion(event.matches);
  });
}

// ========================================
// INITIALISE
// ========================================

body.dataset.orbState = orbState;
body.dataset.paused = "false";
body.dataset.silent = "true";

durationTime.textContent = formatTime(SESSION_LENGTH);

intensityValue.textContent = `${intensityInput.value}%`;
breathValue.textContent = "Slow";

body.style.setProperty("--light-level", "0");
body.style.setProperty("--breath-duration", `${breathDuration}s`);

updateProgress();
updatePlaybackUI();

// The orb breathes independently, even during simulated silence.
orbFrame = requestAnimationFrame(animateOrb);
```